

Creating Song List I

Week 05.



COMT

SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



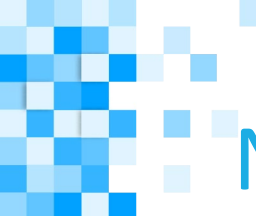
Overview

- You will now start to learn how to code in the course of learning to build a music streaming player mobile app.



Objectives

- Create a simple song list in the music application;
- Demonstrate how to create a song listing UI using Android Studio
- Understand what is a method and parameter.
- Understand what is an event Listener



Note

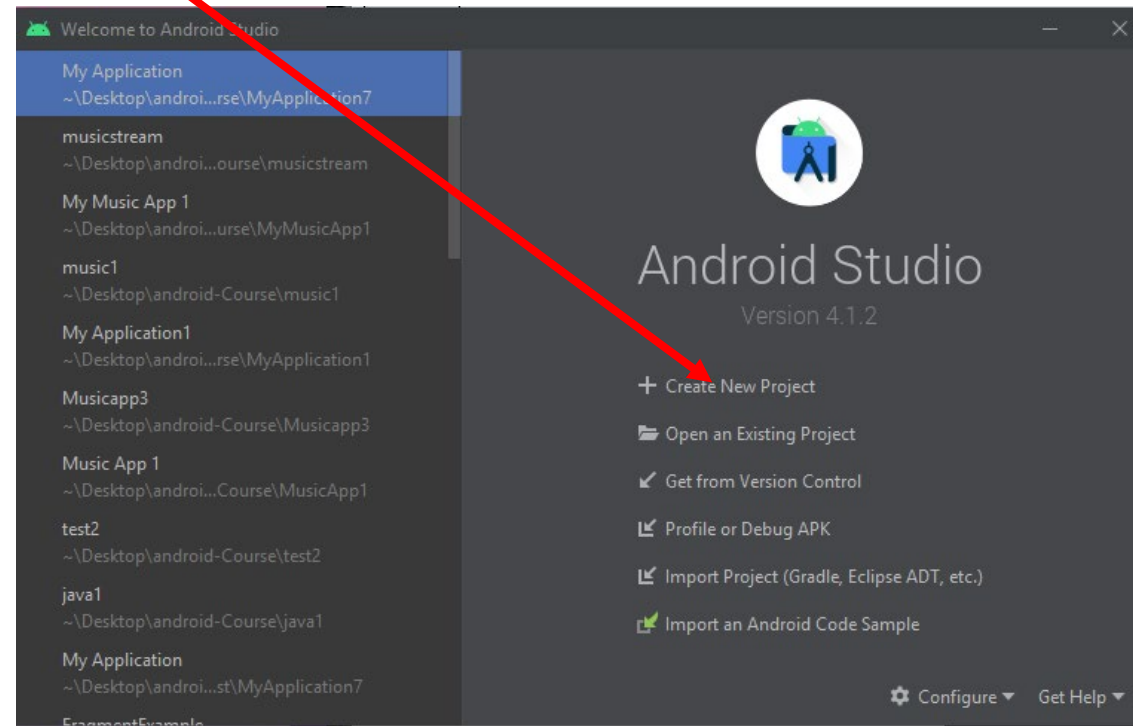
- Throughout this document, the acronym “**app**” will be used to refer to **mobile application**.



CREATE NEW PROJECT

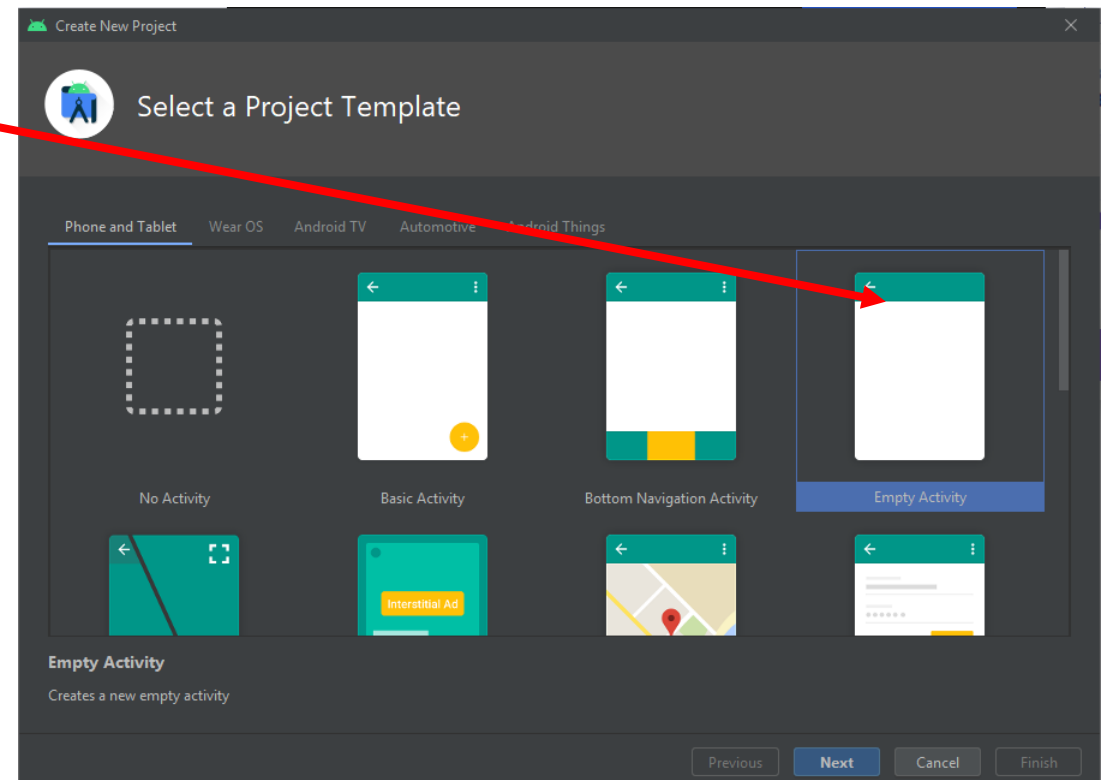
Create New Project

- Open Android Studio.
- choose Create New Project.



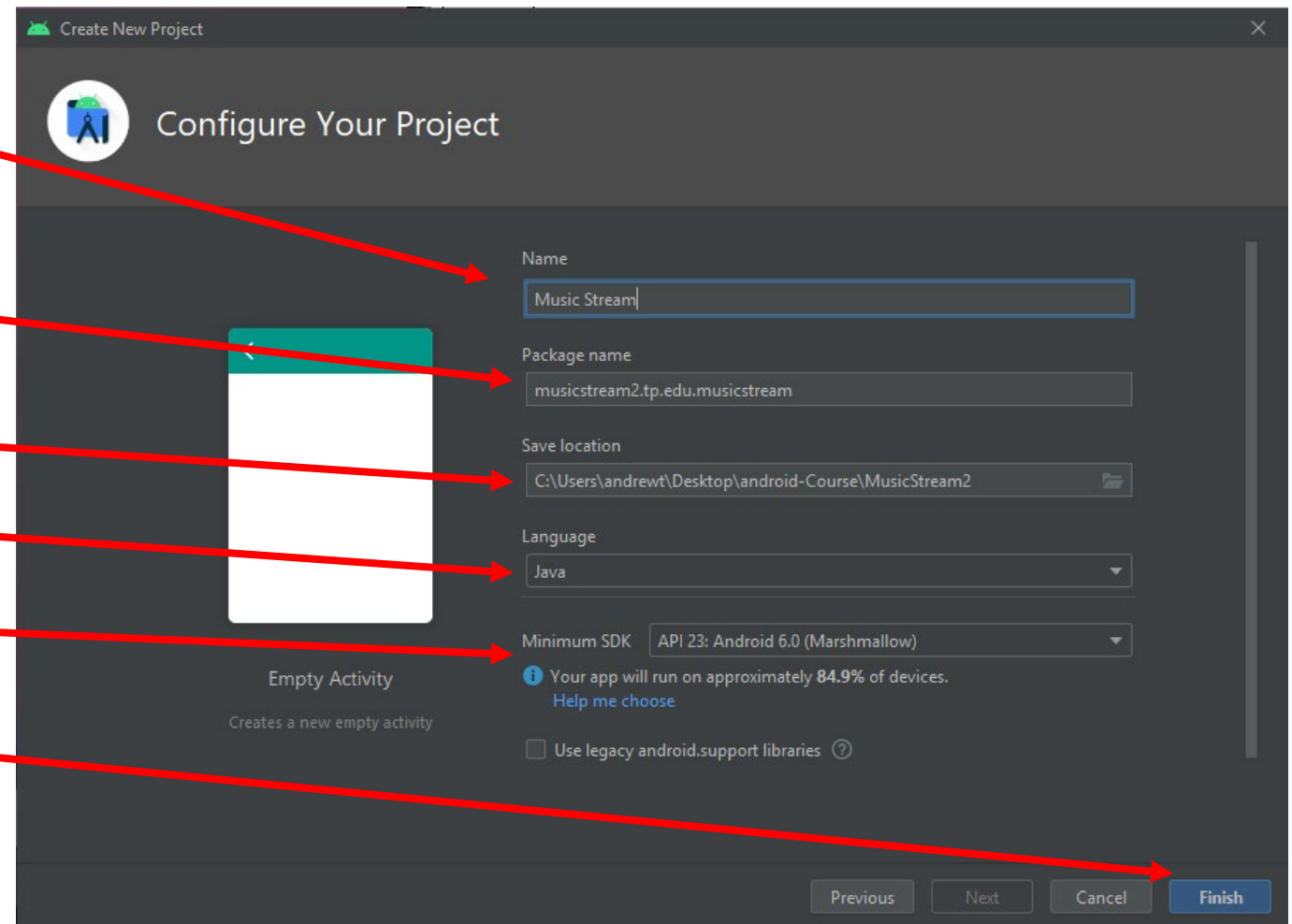
Hello World Project – Step 1: Start & Configure a new project

- Choose **Empty Activity**
- Click **Next**.



Hello World Project – Step 1: Start & Configure a new project

- Name of your app
Music Stream
- Package Name
(Accept suggested name)
- Your **Project Location**
- Make sure it is Java
- Accept the default API 23
- Click finish
and allow Android Studio to
complete the process





EXPLORING ANDROID STUDIO

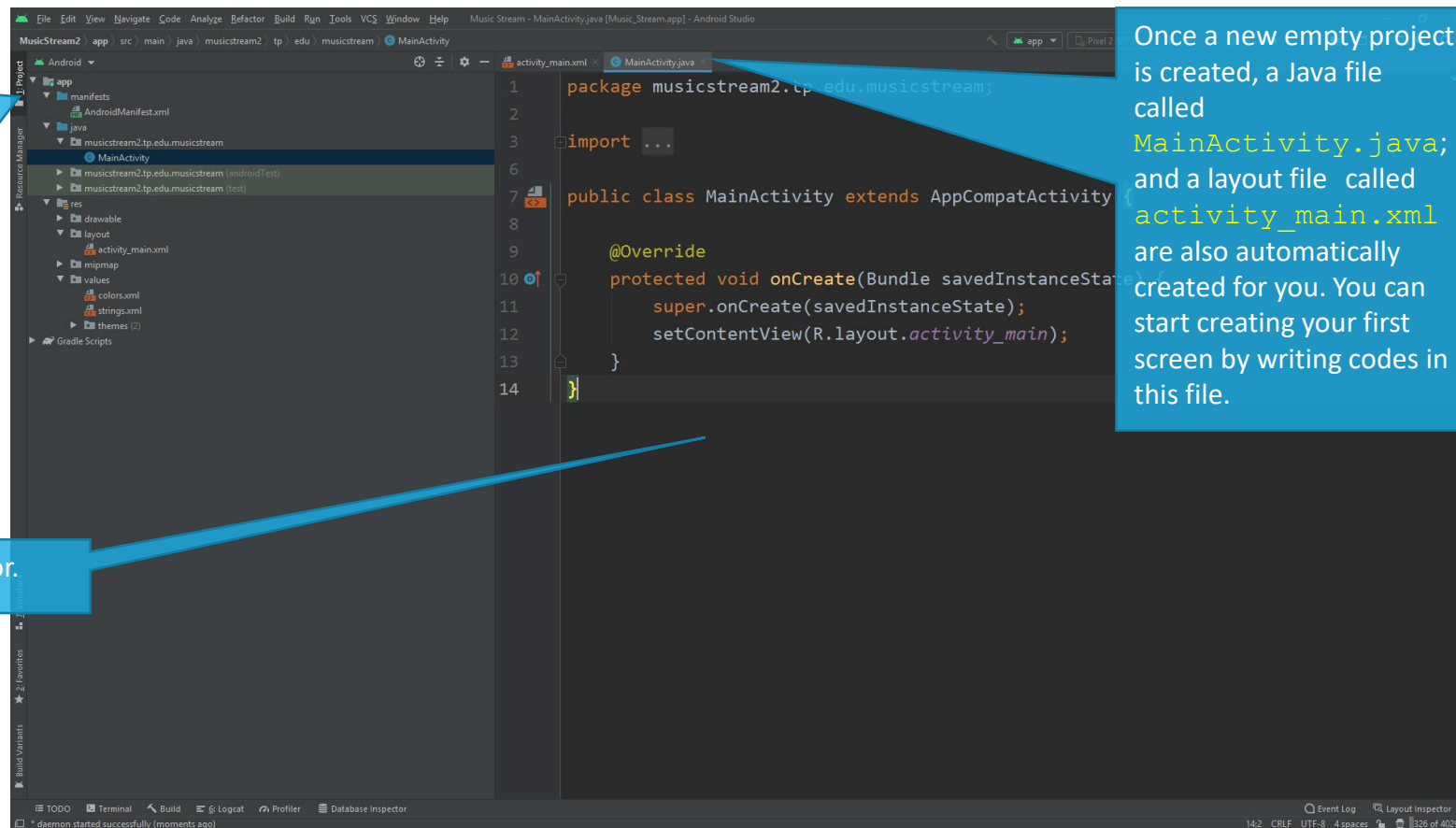
Exploring Android Studio

- Once a new empty project is created, you will see the window below. Take a while to explore this interface where you will be developing your app.

The **Project** tab here shows all the files required for creating the app. You can add new files or modify existing files. The files are organized in different folders.

Go to the next slide to see more explanations of this project structure.

Source code editor.



Exploring Android Studio

1) The **app** folder is the root of all the application files. You can click on the inverted triangle to collapse or open the folder.

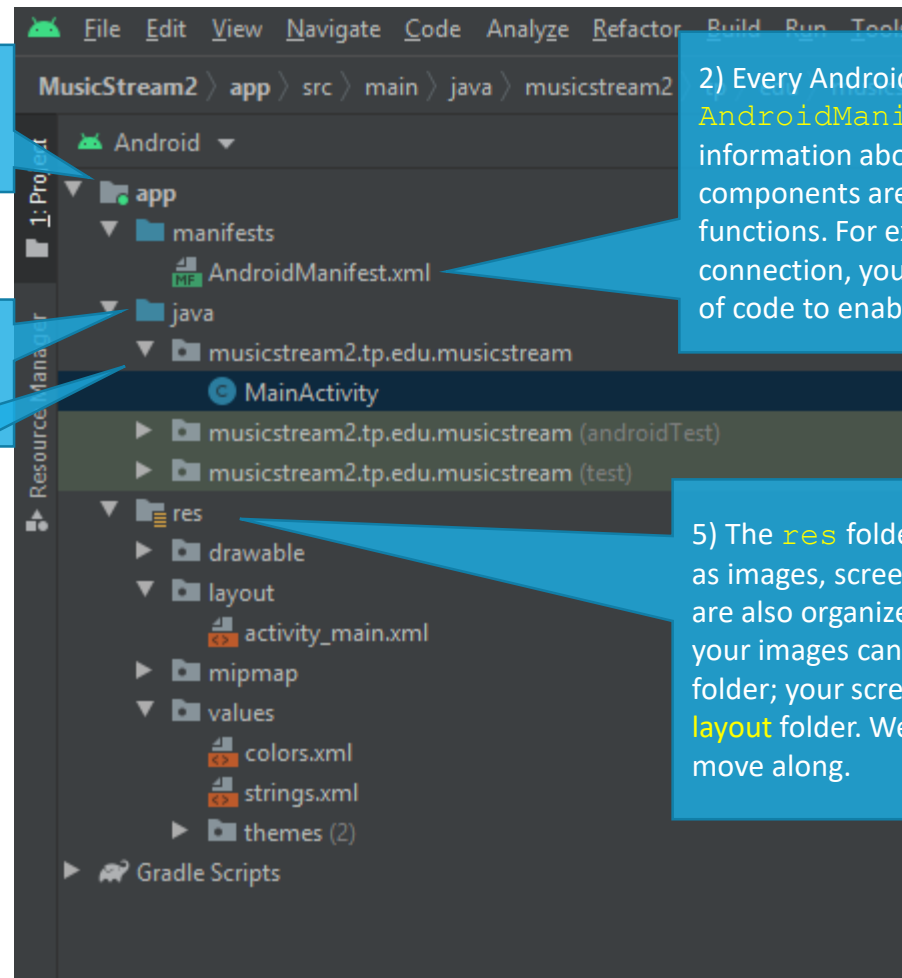
3) The **java** folder contains all Java files that you will be creating as you learn and write codes to build the app.

4) This sub folder called **sg.edu.tp.musicstream** is also known as a “**package**” folder and contains one or many Java files. Notice that it is named based on your company domain but in reverse. This is the naming standard for Java package folders. You can create different package folders to organize your Java files. Let’s create a new package in the next slide!

Note: In this basic course, you can ignore the two package folders with “**(androidTest)**” and “**(test)**”.

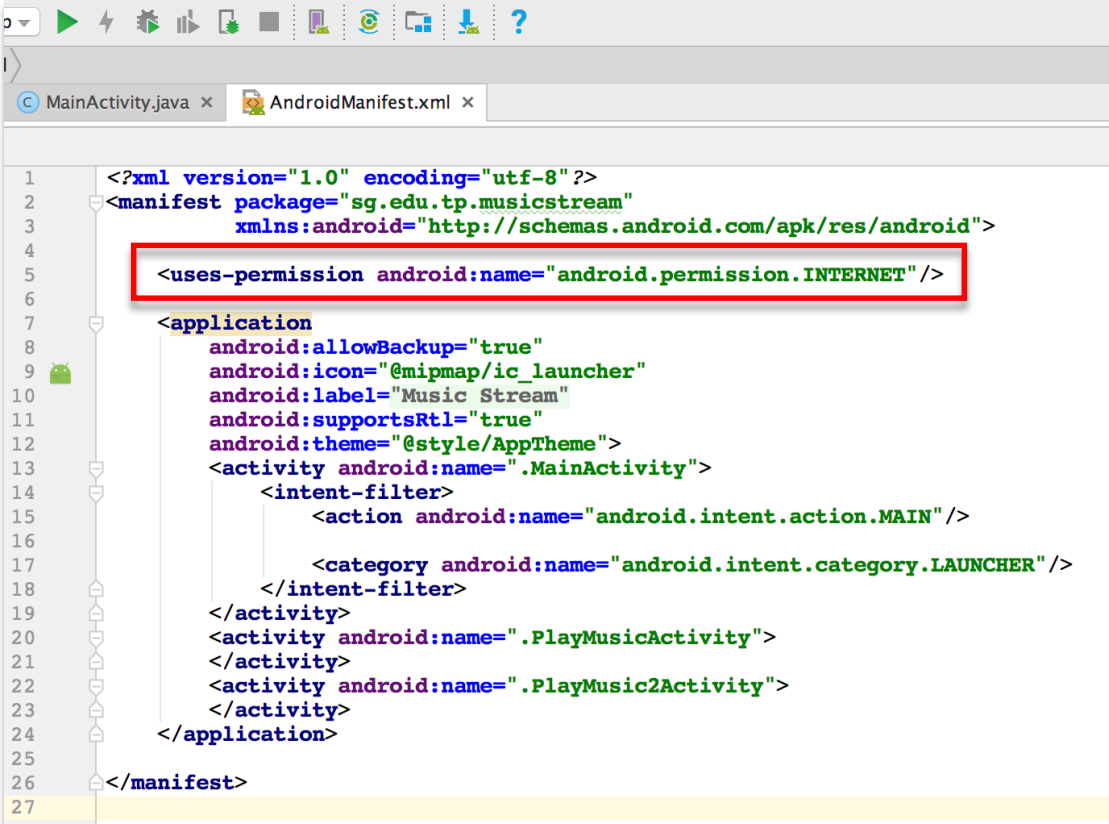
2) Every Android app must contain this **AndroidManifest.xml** file. It provides information about your app such as what components are required to run certain functions. For example, if you require internet connection, you will need to add a particular line of code to enable it.

5) The **res** folder contains all resource files such as images, screens, settings, etc. Each of these are also organized in sub folders. For example, your images can be added to the **drawable** folder; your screen designs can be added into the **layout** folder. We will explain the rest as you move along.



Enabling Internet

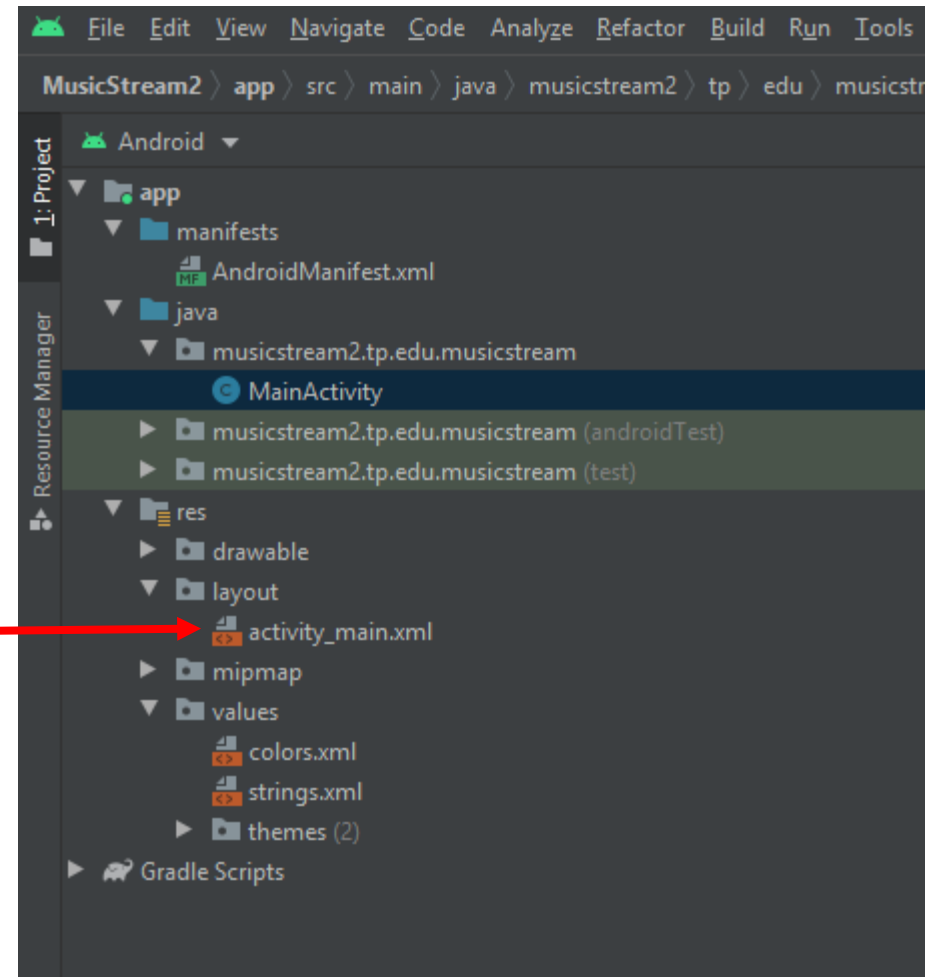
- For the Music Stream app, we need to enable Internet to be able to stream music within the app.
- This is done by adding a line in the **AndroidManifest.xml** file.
- Open the file and add the highlighted line of code shown in the diagram. Add it above the start of **<application** tag.



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest package="sg.edu.tp.musicstream"
3         xmlns:android="http://schemas.android.com/apk/res/android">
4     <uses-permission android:name="android.permission.INTERNET" />
5
6     <application
7         android:allowBackup="true"
8         android:icon="@mipmap/ic_launcher"
9         android:label="Music Stream"
10        android:supportsRtl="true"
11        android:theme="@style/AppTheme">
12        <activity android:name=".MainActivity">
13            <intent-filter>
14                <action android:name="android.intent.action.MAIN"/>
15
16                <category android:name="android.intent.category.LAUNCHER" />
17            </intent-filter>
18        </activity>
19        <activity android:name=".PlayMusicActivity">
20        </activity>
21        <activity android:name=".PlayMusic2Activity">
22        </activity>
23    </application>
24
25 </manifest>
26
27
```

Setting the App Dimension

- There are many sizes of mobile phones and it is always good to choose the most common/popular size to build your app for.
- Before you start creating anything in your app, you need to select the dimension of the app for the size of the mobile phone.
- Double-click the `activity_main.xml` file to open it in **Design** view.

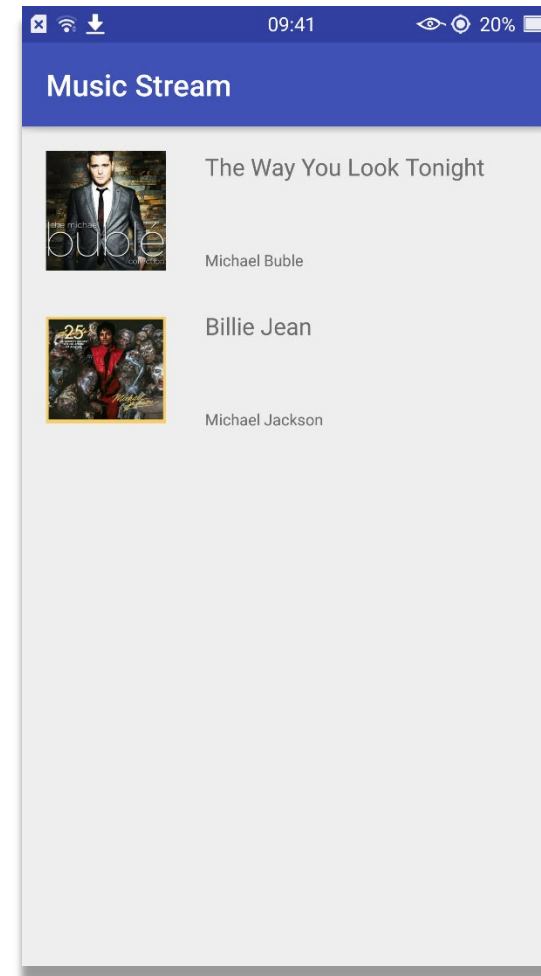




CREATING THE SCREEN UI

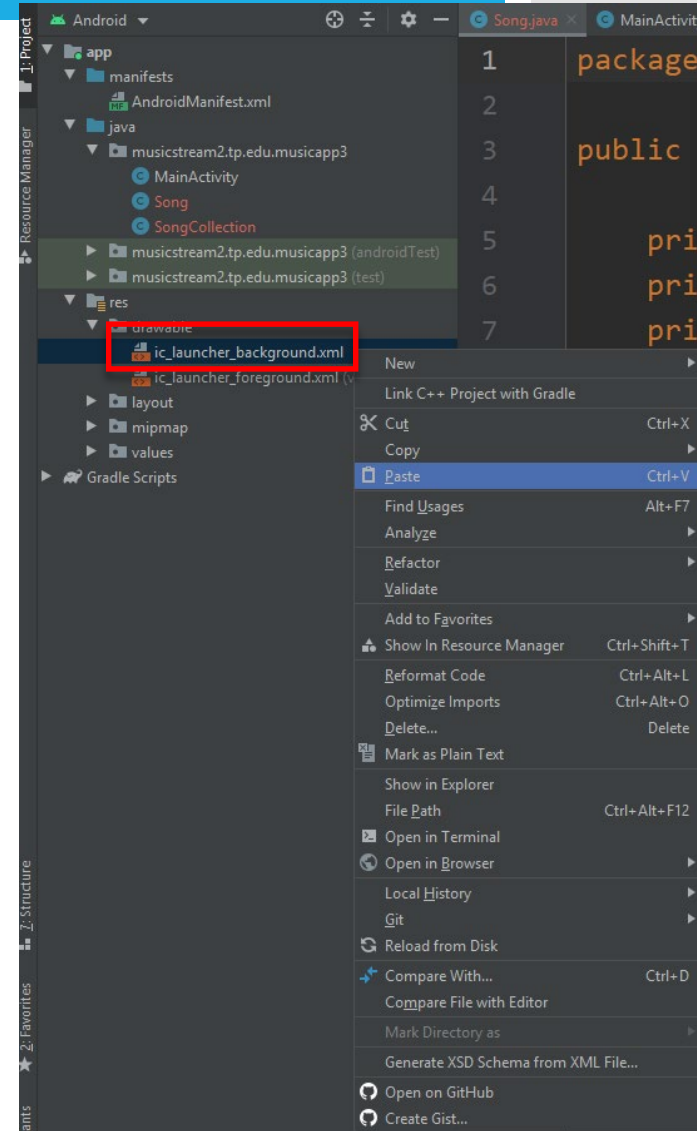
Creating the Song Listing UI

- You will now learn to build a simple song listing as shown in the diagram here.



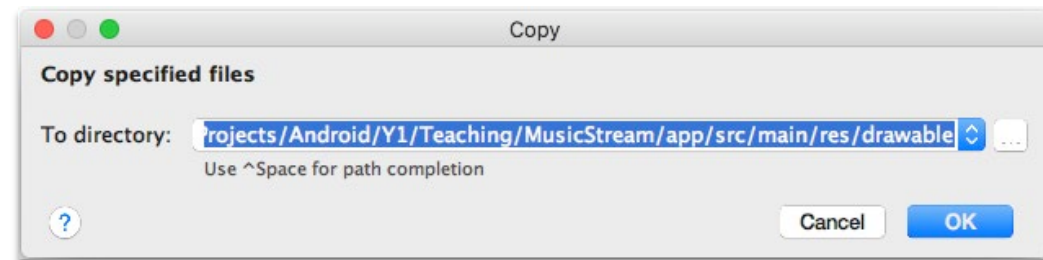
Creating the Song Listing UI

- The first thing you need to do is to download the assets file from LMS.
- Log in to LMS and go to Computational Thinking -> Content -> Week 5 folder.
- Download the **assets.zip** file and save to your desktop.
- Unzip the **assets.zip** file and double click to go into the **assets** folder.
- Copy all image files.
- Go to Android Studio and right-click on the **ic_launcher_background.xml** and select **Paste** as shown in the diagram here. (Do not choose the one with (v24))
- Continue to next slide.



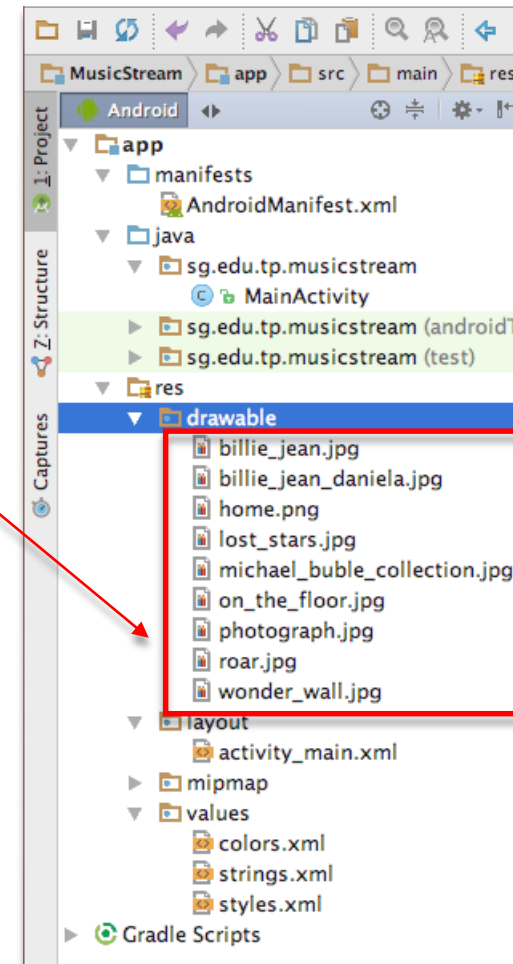
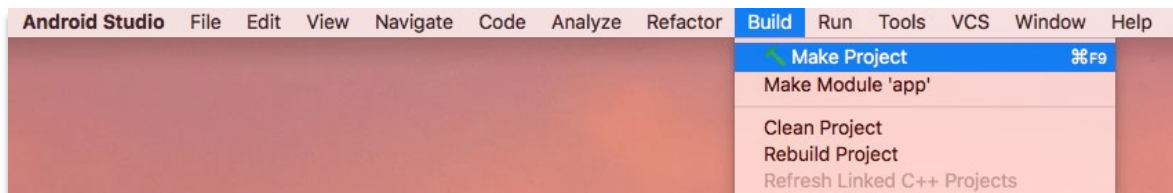
Creating the Song Listing UI

- A dialogue box will appear to prompt if you want to paste to the highlighted directory.
- Just click **OK**.
- Continue to next slide.



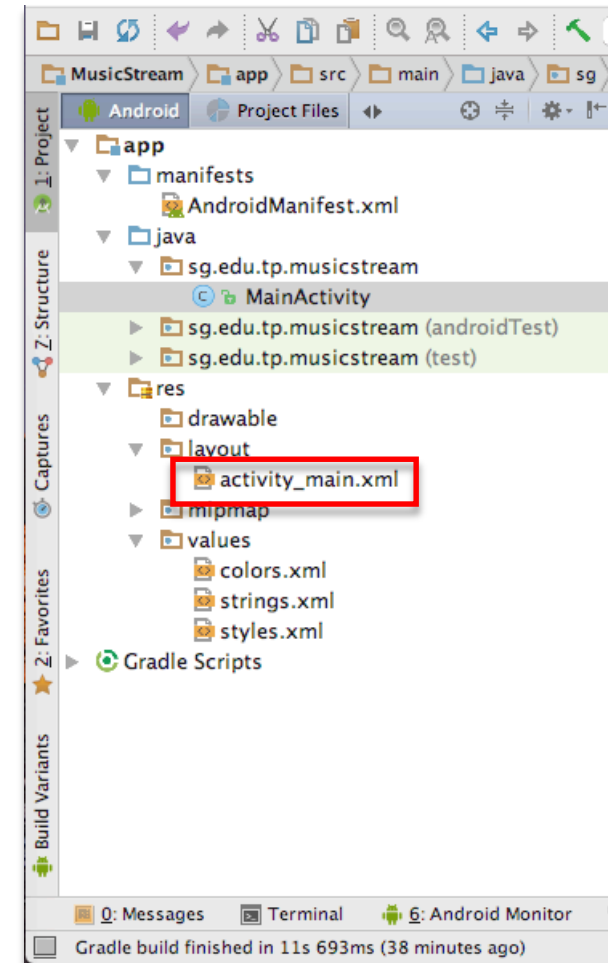
Creating the Song Listing UI

- If all image files are successfully copied, you will see the list of image files as shown in the highlighted area.
- If the `drawable` folder is collapsed, just click on the triangle icon beside it to open.
- At the Android Studio menu as shown below, go to **Build** and select **Make Project**. This will register the newly added images to the system.



Creating the Song Listing UI

- To start creating the UI for the song list, double click on the file `activity_main.xml` under the `layout` folder as highlighted in the image.



Creating the Song Listing UI

2) The **Palette** section here shows a collection of user interface widgets that you can use to build your screen.

3) The **Component Tree** section here shows how the widgets appear on the screen.

In this example, the **TextView** widget is positioned within the **ConstraintLayout** element.

Click on the **TextView** here and go to the next slide.

1) After you double click on the file **activity_main.xml**, it will show a near empty screen by default. The **activity_main** is the first screen to appear on your app.

To add widgets to this screen, just select a widget such as **Button** from the **Palette** section, drag it and drop onto the screen where you want it to be positioned.

Creating the Song Listing UI

These 3 options toggle between the visual GUI and XML view

2) You do not have to be scared by all the text here. It is actually easy to read by recognizing certain keywords for the widget.

In this example, the section here represents the whole `TextView` widget. The `android:text` keyword refers to the text being displayed, which is `"Hello World!"`. Change the text to `"Hello Everyone!"` and then switch to `Design` tab below to see the change.

1) This is the textual representation of the screen UI. It is called an XML file written in XML format.

XML stands for **eXtensible Markup Language** and was designed to store and transport data.

Over here, it is used to define layout of user interface.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res-auto"
3   xmlns:tools="http://schemas.android.com/tools"
4   android:layout_width="match_parent"
5   android:layout_height="match_parent"
6   tools:context=".MainActivity">
7
8   <TextView
9     android:layout_width="wrap_content"
10    android:layout_height="wrap_content"
11    android:text="Hello World!"
12    app:layout_constraintBottom_toBottomOf="parent"
13    app:layout_constraintLeft_toLeftOf="parent"
14    app:layout_constraintRight_toRightOf="parent"
15    app:layout_constraintTop_toTopOf="parent"/>
16
17 </androidx.constraintlayout.widget.ConstraintLayout>
```

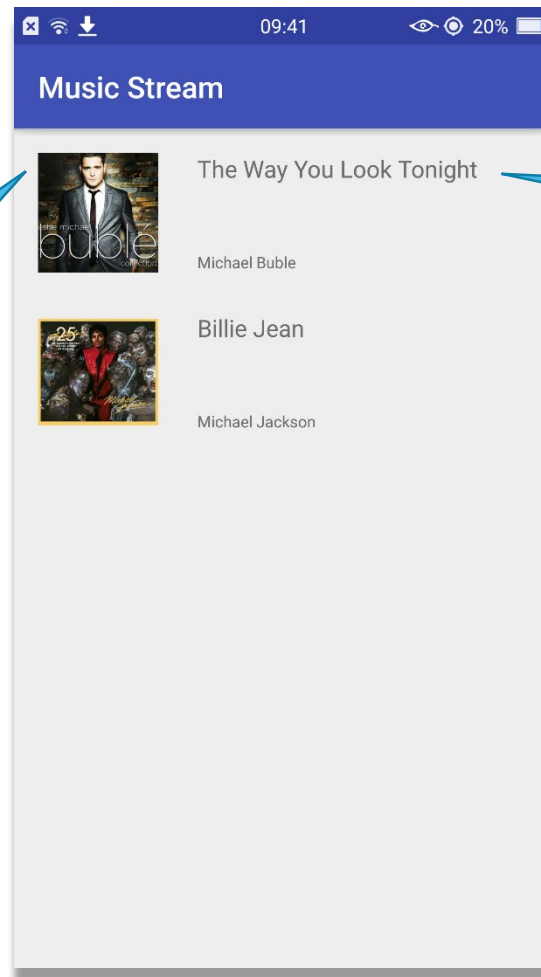
Creating the Song Listing UI

1) After practising dragging and dropping the widgets, you can now start to create the song list UI!

2) Use `ImageButton` for album thumbnails.

The reason for using `ImageButton` is to allow a user to tap on the image to select the song.

The `ImageButton` has the effects of a button.



3) Use `TextView` widget for song title and artist name.

4) Your lecturer will now demonstrate to you how to add the first song in the list.

The properties for each item of the song are shown in the next slide.

Do watch closely and ask questions if you are in doubt.

You can also refer to the online video

Creating the Song Listing UI

- After watching the demonstration by your lecturer, it is now your turn to add a new song to the list.
- Use the following information to help you add the new song.

Thumbnail (**ImageButton**)

No.	Properties	Value
1.	ID	S1001
2.	layout_width	80dp
3.	layout_height	80dp
4.	scaleType	centerCrop
5.	onClick	handleSelection

Song Title (**TextView**)

No.	Properties	Value
1.	ID	txtTitle1
2.	layout_width	wrap_content
3.	layout_height	wrap_content
4.	text	The Way You Look Tonight
5.	textSize	18sp

Artist Name (**TextView**)

No.	Properties	Value
1.	ID	txtArtist1
2.	layout_width	wrap_content
3.	layout_height	wrap_content
4.	text	Michael Buble
5.	textSize	12sp

Creating the Song Listing UI

- After watching the demonstration by your lecturer, it is now your turn to add a new song to the list.
- Use the following information to help you add the new song.

Thumbnail (ImageButton)

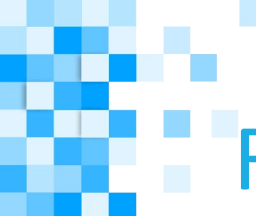
No.	Properties	Value
1.	ID	S1002
2.	layout_width	80dp
3.	layout_height	80dp
4.	scaleType	centerCrop
5.	onClick	handleSelection

Song Title (TextView)

No.	Properties	Value
1.	ID	txtTitle2
2.	layout_width	wrap_content
3.	layout_height	wrap_content
4.	text	Billie Jean
5.	textSize	18sp

Artist Name (TextView)

No.	Properties	Value
1.	ID	txtArtist2
2.	layout_width	wrap_content
3.	layout_height	wrap_content
4.	text	Michael Jackson
5.	textSize	12sp



Run It!

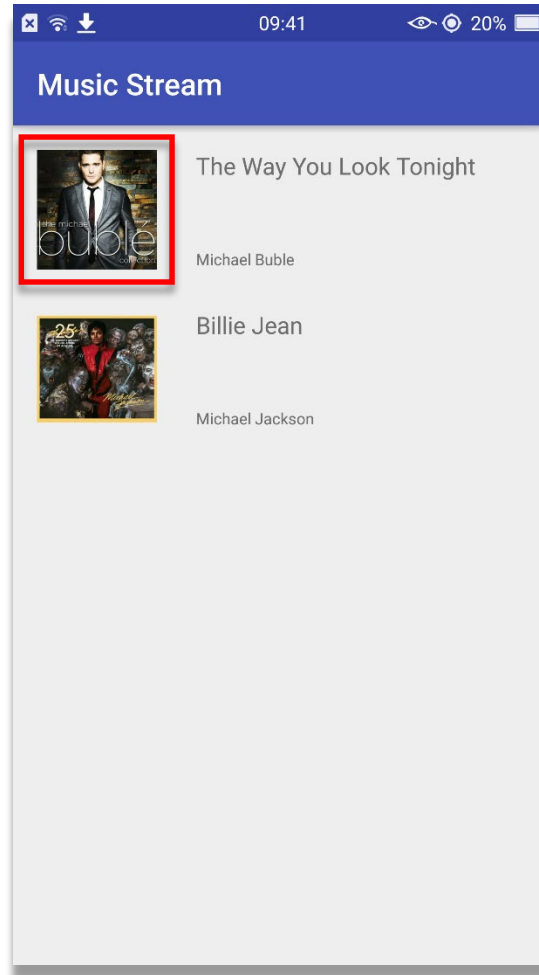
- After creating the songs on the screen, you can now run your app and see it on the emulator!
- Click the Run button in Android Studio and watch your newly created screen on the emulator!



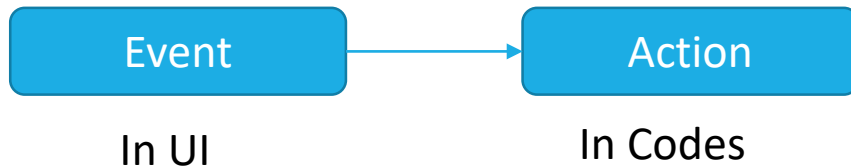
HOW TO SELECT A SONG AND DO SOMETHING

How to Select a Song from the UI?

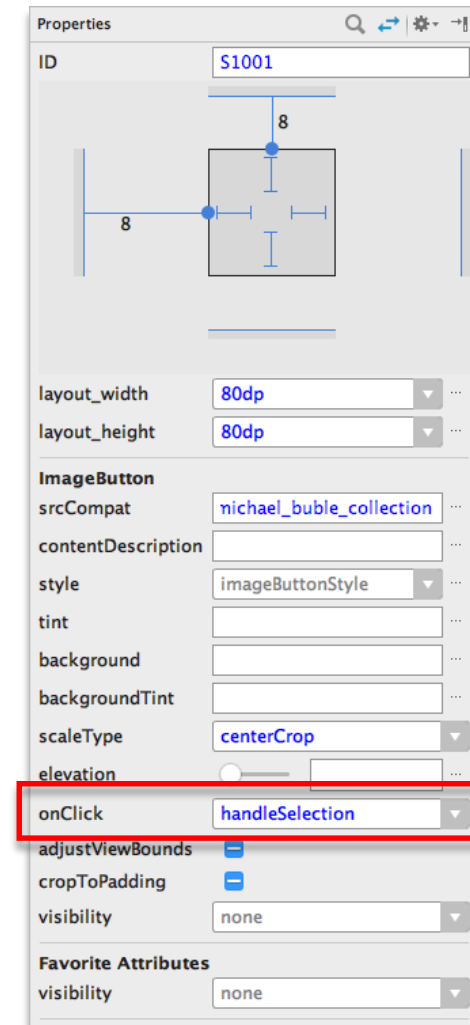
- As mentioned in the previous section, we use the `ImageButton` for the thumbnail so that a user can tap on the thumbnail to select it.
- An `ImageButton` is able to link an event to an action that we create through codes.
- For example, we can write codes to change to a new screen to show a player with the selected song when a user taps on a thumbnail in the song list screen.
- Let's create the action by writing some codes as explained in the next few slides.



How to Select a Song from the UI?

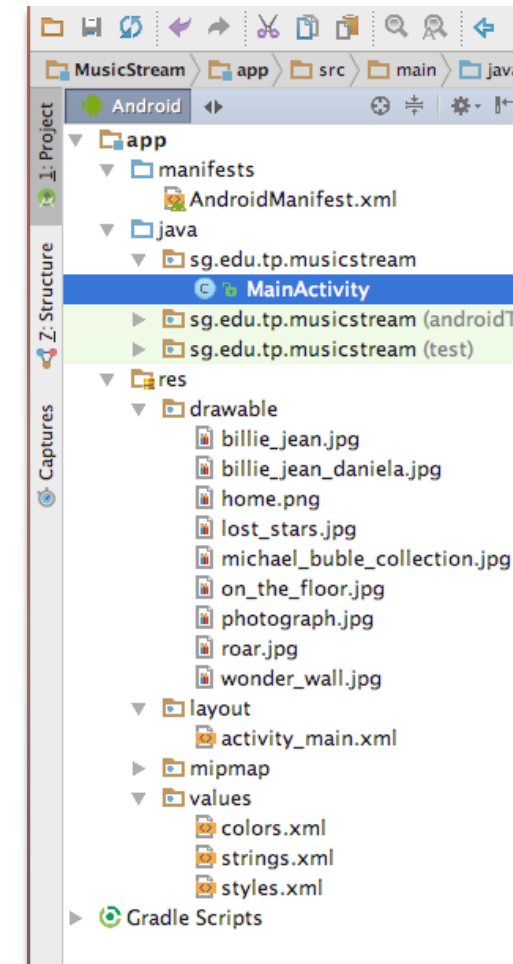


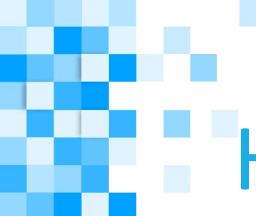
- When we create an action for an event, we need to link the event in the UI to the action, which consists of written codes.
- This event in the UI is called **onClick** as highlighted in the diagram.
- Notice that you have already entered a name called “**handleSelection**”. This will be the name of the action that you will be creating in the codes.
- Go to the next slide to create the action.



How to Select a Song from the UI?

- Double-click on **MainActivity** to open the `MainActivity.java` file for editing as shown in the diagram.





How to Select a Song from the UI?

- Before you start writing codes, let's look at the steps on the right that you need to accomplish for the `handleSelection` action.
- When you write these steps out, you are writing what is known as **Pseudocode**. A **Pseudocode** is a descriptive form of writing the steps that will later be written as codes.
- Your tutor will show you how to convert each step into codes and also the importance of logging output in the console

1. Each ImageButton will call the `handleSelection` method.
2. Figure out the Id of the ImageButton that was pressed
3. Log the Id onto the Android Studio console

How to Select a Song from the UI?

The screenshot displays the Android Studio interface. The top editor shows the `MainActivity.java` file with the following code:

```
package sg.edu.tp.musicstream;

import ...

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void handleSelection(View myView) {
        String resourceId = getResources().getResourceEntryName(myView.getId());
        Log.d(tag, "temasek", msg: "The id of the pressed ImageButton is : " + resourceId);
    }
}
```

Annotations on the code:

- A blue callout points to the `handleSelection` method: "Whenever any of the ImageButton is being tapped, it will call the handleSelection method."
- A blue callout points to the `getResourceEntryName` call: "This code will be able to figure out what is the string ID of the ImageButton"
- A blue callout points to the `Log.d` statement: "Log to the console Logcat so that we can trouble-shoot."

The Logcat window at the bottom shows the following output:

```
2021-04-02 22:13:23.564 6948-6948/sg.edu.tp.musicstream D/temasek: The id of the pressed ImageButton is :S1002
2021-04-02 22:13:46.899 6948-6948/sg.edu.tp.musicstream D/temasek: The id of the pressed ImageButton is :S1001
2021-04-02 22:13:48.048 6948-6948/sg.edu.tp.musicstream D/temasek: The id of the pressed ImageButton is :S1003
```

The mobile emulator on the right shows a UI titled "COMT" with a list of songs:

- The Way You Look Tonight - Michael Buble
- Billie Jean - Michael Jackson
- One - Ed Sheeran

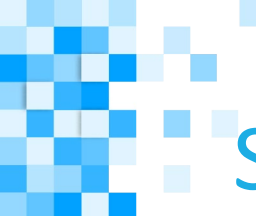
Annotations on the Logcat output:

- A blue callout points to the log messages: "Each logged statement will be associated with the word 'Temasek' so that we can filter it from the rest"
- A red box highlights the "Logcat" tab in the bottom toolbar.



Summary

- You have now successfully created the UI for the song listings!
- You have also written intelligent codes to console out the Id of the ImageButton that was pressed – VERY IMPT!
- In the next lesson, you will learn how to create a song list that stores different information of each song.



Supplementary Materials

Topic	Source
Android UI	https://www.youtube.com/watch?v=z53Ed0ddxgM
	https://www.youtube.com/watch?v=CTYf1qwrXuW
	https://developer.android.com/guide/topics/ui/index.html
Pseudocode	http://www.unf.edu/~broggio/cop2221/2221pseu.htm
	https://en.wikipedia.org/wiki/Pseudocode
	https://www.khanacademy.org/computing/computer-programming/programming/good-practices/p/planning-with-pseudo-code

THE END



COMT